Fatec Jales Prof. José Camargo | Centro Paula Souza | GOVERNO DO ESTADO DE SÃO PAULO

Faculdade de Tecnologia Professor José Camargo

Sprint Documentation		
Sprint 18	**Start Date: 08/11/2026**	**Final Date: 11/11/2026**
Project Name:	**civitas-backend**	
Ano: 5º	**Análise e Desenvolvimento de Sistemas - AMS**	

Team Members
Gustavo Henrique Moreira Porto
Wendell Alves Nascimento

Sprint Backlog			
Task #	Description	Start Date	Final date
#111	Implementação de WebSocket para Auditoria	19/05/2026	22/05/2026

Task Description					
Task #	Description	Assigned To	Status	Estimated Hours	Logged Hours
#111	Implementação do SignalR para Auditoria em Tempo Real	Wendell Alves Nascimento e Gustavo Henrique Moreira Porto.	Completo	10 horas	10 horas

Sprint Description

Tarefa: Implementação do SignalR para Auditoria em Tempo Real

Objetivo

Implementar a comunicação em tempo real entre o backend **C# / ASP.NET Core** e o frontend utilizando **ASP.NET Core SignalR**, permitindo que usuários autorizados recebam notificações sobre novos registros de auditoria sem precisar atualizar a página.

A implementação deve **reutilizar o módulo de auditoria existente** e disparar eventos somente após a confirmação da persistência do respectivo registro. **Não faz parte desta task** criar o sistema de auditoria, alterar suas regras de validação ou implementar a interface visual do frontend.

Descrição

Adicionar ao backend um canal de comunicação SignalR para publicar eventos de novas auditorias. Quando uma operação auditável for concluída e seu registro de auditoria for efetivamente salvo, o sistema deverá emitir um evento aos clientes conectados e autorizados.

O payload enviado deverá conter apenas os dados necessários para atualização da interface. Os detalhes completos e snapshots da auditoria deverão continuar disponíveis por endpoints de consulta protegidos, respeitando as regras de autorização.

A integração não poderá comprometer a operação principal nem tornar a persistência de auditorias dependente da disponibilidade do frontend ou da conexão SignalR.

Critérios de Aceite

◈ Configuração do SignalR

- Adicionar e configurar o SignalR no ASP.NET Core, reutilizando os pacotes do framework quando disponíveis.

Registrar os serviços necessários no container de injeção de dependências (AddSignalR).

Mapear um endpoint de Hub, por exemplo MapHub<AuditoriaHub>("/hubs/auditoria").

Garantir que o Hub esteja acessível somente por conexões autorizadas.

Configurar CORS para as origens permitidas do frontend, caso esteja hospedado em domínio diferente.

Configurar corretamente autenticação e transporte da conexão (incluindo WebSockets quando disponíveis).

Evitar expor o Hub publicamente sem controle de acesso.

◈ Criação do Hub

- Criar a classe AuditoriaHub derivada de Hub.

Aplicar autorização ao Hub com [Authorize] e políticas ou permissões específicas para leitura de auditoria.

Não permitir que clientes publiquem livremente registros ou eventos de auditoria.

Manter o Hub responsável pela comunicação, sem deslocar para ele as regras de negócio ou persistência.

Implementar métodos de grupos/assinaturas somente se houver necessidade de segmentação por organização, unidade ou permissão.

Garantir que qualquer entrada usada para inscrição em grupos seja validada e autorizada pelo servidor.

🔷 Evento de Nova Auditoria

- Definir o nome padronizado do evento enviado ao frontend, por exemplo AuditoriaCriada.

Criar um DTO específico para notificação, por exemplo AuditoriaNotificacaoDto.

    - Incluir no payload somente os campos necessários para atualizar a listagem, como:
        - id
        - operacao
        - nomeEntidade
        - `ValoresAtingidos
        - 'NovosValores'
        - data (com fuso ou UTC explicitamente definido)
        - 'hora'
        - usuarioId, somente se autorizado e necessário

Não enviar snapshots completos (ValoresAtingidos e NovosValores) por padrão.

Não incluir senhas, tokens, credenciais ou dados pessoais desnecessários no evento.

Padronizar o contrato do evento para integração com o frontend.

Garantir que o evento represente exclusivamente registros novos efetivamente persistidos.

Exemplo sugerido de payload:

{

"id": 125,

"operacao": "UPDATE",

"nomeEntidade": "Fornecedor",

"data": "2026-11-07",

"hora": "10:01:44",

"`ValoresAtingidos": "Papelaria Rufus",

"NovosValores": "Papelaria Bufus"

"usuarioId": "2551"

}

O exemplo é ilustrativo. O formato final deverá ser compatível com a modelagem e com o padrão temporal adotado no projeto.

◈ Integração com o Fluxo de Auditoria

- Identificar o ponto central de criação e persistência dos registros de auditoria.

Integrar o disparo das notificações ao fluxo existente, evitando duplicação em vários controllers.

Utilizar IHubContext<AuditoriaHub> (ou uma abstração de publicação) para enviar eventos a partir do backend.

Publicar a notificação **somente depois de confirmada a persistência** da auditoria e, quando aplicável, o commit da transação.

Não publicar eventos para registros rejeitados por validação ou operações que falharam.

Evitar o disparo duplicado do mesmo evento dentro de uma mesma execução.

Garantir que falhas na transmissão em tempo real não desfaçam operações de negócio já confirmadas.

Registrar falhas técnicas de publicação em logs internos, sem expor informações sensíveis.

◈ Autenticação e Autorização

- Integrar a conexão SignalR ao mecanismo de autenticação já existente no projeto.

Aplicar a política/permissão de acesso aos registros de auditoria.

Impedir que usuários comuns recebam eventos restritos a administradores ou auditores.

Se houver separação por organização, unidade ou tenant, impedir recebimento de eventos de outros contextos.

Validar permissões antes de inscrever conexões em grupos de transmissão.

Revisar comportamento quando a sessão expirar ou o acesso for revogado, inclusive em conexões já abertas.

Evitar incluir tokens de acesso em URLs ou logs de forma insegura; caso o transporte exija token em query string, aplicar as proteções apropriadas.

◈ Conexões e Reconexões

- Permitir múltiplos clientes autorizados conectados simultaneamente.

Garantir que desconexões não gerem falha no fluxo principal de auditoria.

Definir comportamento esperado de reconexão do frontend.

Documentar que SignalR não garante a entrega de eventos a clientes desconectados.

Permitir que o frontend recupere eventos perdidos por meio da API de consulta, por exemplo utilizando o último ID recebido ou recarregando a listagem.

Não depender do SignalR como única fonte do histórico de auditoria.

◈ Tratamento de Erros e Observabilidade

- Tratar falhas de conexão e de publicação sem interromper as operações principais.

Registrar erros técnicos e informações de diagnóstico nos logs do backend.

Não retornar stack traces ou informações internas aos clientes.

Evitar logar payloads sensíveis ou credenciais de autenticação.

Disponibilizar mensagens e eventos com nomenclatura padronizada.

Garantir que a aplicação continue operando quando não houver clientes conectados.

🧭 Contrato para o Frontend

- Documentar a URL do Hub, por exemplo /hubs/auditoria.

Documentar o nome do evento, por exemplo AuditoriaCriada.

Documentar o DTO/payload e os tipos dos campos.

Documentar os requisitos de autenticação e autorização.

Documentar a estratégia de reconexão e sincronização de eventos perdidos.

Disponibilizar exemplo de consumo com @microsoft/signalr, caso o frontend utilize JavaScript ou TypeScript.

Validar a integração com um cliente de teste ou frontend de desenvolvimento.

Ajustes Técnicos Esperados

- Criar AuditoriaHub.

Criar AuditoriaNotificacaoDto.

Configurar AddSignalR() e MapHub<AuditoriaHub>(...) no ponto de inicialização da aplicação.

Criar, se necessário, IAuditoriaNotificacaoService e AuditoriaNotificacaoService para centralizar o envio.

Integrar a publicação ao serviço/fluxo que persiste as auditorias.

Reutilizar a autenticação existente (JWT/cookies, conforme o projeto).

Configurar políticas de autorização e, quando necessário, grupos de acesso.

Configurar CORS para as origens confiáveis do frontend.

Adicionar logs técnicos de falhas de notificação.

Documentar o contrato de comunicação com o frontend.

Não criar migration, a menos que alguma necessidade adicional de persistência seja identificada e justificada.

Testes Automatizados

Configuração e Segurança

- Testar conexão com usuário autenticado e autorizado.

Testar rejeição de conexão não autenticada.

Testar rejeição de usuário sem permissão para consultar auditorias.

Testar isolamento entre grupos, unidades ou tenants, se aplicável.

Testar que clientes não podem fabricar eventos de auditoria.

Publicação de Eventos

- Testar envio do evento quando uma auditoria for persistida com sucesso.

Testar ausência de evento quando a validação da auditoria falhar.

Testar ausência de evento quando a transação não for confirmada.

Testar que o payload corresponde ao contrato documentado.

Testar que o payload não contém snapshots completos ou dados sensíveis não autorizados.

Testar que múltiplos clientes autorizados recebem o evento.

Testar que usuários não autorizados não recebem o evento.

Resiliência

- Testar funcionamento da aplicação sem clientes conectados.

Testar falha de publicação sem desfazer a persistência já concluída.

Testar desconexão e reconexão de clientes.

Testar recuperação de registros perdidos por consulta à API.

Testar que não há disparos duplicados no fluxo normal de publicação.

Observações e Sugestões

- **Esta task contempla o backend SignalR e a validação da integração.** A construção da interface visual e o gerenciamento da conexão na aplicação frontend podem constituir uma task separada.
- Preferir IHubContext<AuditoriaHub> ao envio diretamente por instâncias do Hub.
- O registro no banco de dados é a fonte oficial do histórico; o SignalR serve para avisar sobre novidades.
- Não transmitir a auditoria antes da confirmação da transação.
- Para projetos com múltiplas instâncias de backend, avaliar infraestrutura de escalabilidade do SignalR (por exemplo, backplane ou serviço gerenciado).
- O envio por IHubContext imediatamente após o commit é uma solução inicial simples, mas não garante publicação em caso de queda do processo entre o commit e o envio. Se houver necessidade de entrega mais confiável, avaliar o padrão **Transactional Outbox** em uma task futura.
- Avaliar limitação de frequência e agrupamento de eventos caso o sistema produza grande volume de auditorias.
- Verificar o projeto antes da implementação para manter seus padrões de arquitetura e segurança.
- Conversar com a equipe de validação de auditoria para garantir que alterações nessa task e na task de validação não altere os resultados esperados em ambas as tasks e não atrapalhe o seguimento do sistema. Não confie cegamente na IA.
- Autorizado realizar ações diferente das citadas na task mas sem alterar o resultado final de maneira severa e que venha causar problemas no sistema.

Resultado Esperado

Ao concluir a task, o backend deverá notificar, em tempo real e de forma segura, os clientes autorizados sempre que um **novo registro de auditoria for persistido**, permitindo a atualização da interface sem recarregar a página.

A publicação deverá respeitar autenticação, autorização, minimização de dados e isolamento de acesso, sem comprometer a persistência das auditorias ou a execução das operações principais em caso de falha na comunicação em tempo real.

Sprint Results

modificacoes

Classes adicionadas

- `AuditoriaHub`: Hub SignalR protegido pela política de leitura de auditoria.

- `AuditoriaNotificacaoDTO`: contrato enxuto do evento, sem snapshots nem dados do usuário.

- `IAuditoriaNotificacaoService` e `AuditoriaNotificacaoService`: abstração e publicação do evento `AuditoriaCriada`; falhas no envio são registradas sem afetar a persistência.

- `AuditoriaReadRequirement` e `AuditoriaReadHandler`: exigem usuário autenticado com perfil `ADMINISTRADOR`.

- `AuditoriaSignalRTests`: cobre autorização, payload, persistência e falhas de publicação.

Classes editadas

- `AuditoriaService`: publica a notificação somente depois que o repositório confirma a gravação.

- `DevBypassAuthorizationHandler`: mantém a política do Hub fora do bypass de desenvolvimento.

resumo:

Resumo: Hub protegido para administradores, notificação enviada após persistência, payload sem snapshots, CORS e autenticação JWT configurados, documentação e testes automatizados adicionados.

Assinaturas

gov.br
Documento assinado digitalmente
GUSTAVO HENRIQUE MOREIRA PORTO
Data: 09/11/2026 22:04:40-0300
Verifique em https://validar.iti.gov.br

gov.br
Documento assinado digitalmente
WENDELL ALVES NASCIMENTO
Data: 09/11/2026 22:17:07-0300
Verifique em https://validar.iti.gov.br